Class practical

1. Regular Expressions in Python

```
import re

text = "Python is easy to learn. Python is powerful."

# Search for a word
result = re.search("Python", text)
if result:
    print("Pattern found:", result.group())
else:
    print("Pattern not found")

# Find all occurrences of a word
matches = re.findall("Python", text)
print("All matches:", matches)

# Match a pattern at the beginning
result = re.match("Python", text)
if result:
    print("Match at beginning:", result.group())

# Replace a word
new_text = re.sub("Python", "Java", text)
print("After replacement:", new_text)

# Find all numbers
text2 = "I have 2 books and 5 pens."
numbers = re.findall(r'\d+', text2)
print("Numbers found:", numbers)
```

```
Pattern found: Python
All matches: ['Python', 'Python']
Match at beginning: Python
After replacement: Java is easy to learn. Java is powerful.
Numbers found: ['2', '5']
```

2. Finite State Automaton to Recognize Strings Ending with 'ab'

```
def accepts_string(string):
    state = 0

    for char in string:
        if state == 0:
            if char == 'a':
                state = 1
            else:
                state = 0

        elif state == 1:
            if char == 'a':
                state = 1
            elif char == 'b':
                state = 2
            else:
                state = 0

        elif state == 2:
            if char == 'a':
                state = 1
            elif char == 'b':
                state = 0
            else:
                state = 0

    return state == 2


# Test the automaton
strings = ["ab", "aab", "abab", "abb", "abc", "ba"]

for string in strings:
    if accepts_string(string):
        print(string, "-> Accepted")
    else:
        print(string, "-> Rejected")
```

```
ab -> Accepted
aab -> Accepted
abab -> Accepted
abb -> Rejected
abc -> Rejected
ba -> Rejected
```

3. Morphological Analysis Using NLTK

```
import nltk
from nltk.stem import WordNetLemmatizer

# Download WordNet data
nltk.download('wordnet')

# Create a lemmatizer
lemmatizer = WordNetLemmatizer()

# List of words
words = ["cars", "playing", "studies", "running", "better"]

print("Morphological Analysis:")

for word in words:
    if word == "playing" or word == "running":
        lemma = lemmatizer.lemmatize(word, pos='v')
    elif word == "better":
        lemma = lemmatizer.lemmatize(word, pos='a')
    else:
        lemma = lemmatizer.lemmatize(word, pos='n')

    print(word, "->", lemma)
```

```
Morphological Analysis:
cars -> car
playing -> play
studies -> study
running -> run
better -> good
```

4. Finite-State Machine for Morphological Parsing (Plural Formation)

```
def make_plural(noun):
    # Rule 1: Words ending in s, sh, ch, x, or z
    if noun.endswith(('s', 'sh', 'ch', 'x', 'z')):
        return noun + 'es'

    # Rule 2: Consonant + y becomes ies
    elif noun.endswith('y') and len(noun) > 1 and noun[-2].lower() not in 'aeiou':
        return noun[:-1] + 'ies'

    # Rule 3: Default rule
    else:
        return noun + 's'


# Test the morphological parser
nouns = ["cat", "bus", "box", "baby", "class", "book"]

print("Plural Forms:")

for noun in nouns:
    print(noun, "->", make_plural(noun))
```

```
--------------------
Plural Forms:
cat -> cats
bus -> buses
box -> boxes
baby -> babies
class -> classes
book -> books
```

5. Porter Stemmer Algorithm in Python

```
from nltk.stem import PorterStemmer

# Create a Porter Stemmer object
stemmer = PorterStemmer()

# List of words
words = [
    "playing",
    "played",
    "plays",
    "running",
    "runner",
    "studies",
    "studying",
    "connected"
]

print("Porter Stemming:")

for word in words:
    stem = stemmer.stem(word)
    print(word, "->", stem)
```

```
Porter Stemming:
playing -> play
played -> play
plays -> play
running -> run
runner -> runner
studies -> studi
studying -> studi
connected -> connect
```